Ders 2 Genişletilmiş Notu: Abstract Factory Kalıbı ve Switch-Case, Kalıtım ve Kalıp Çözümlerinin Karşılaştırılması

Ders 2 Genişletilmiş Notu: Abstract Factory Kalıbı ve Switch-Case, Kalıtım ve Kalıp Çözümlerinin Karşılaştırılması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kısa Tekrar ve Dersin Düzeni
- 2. Abstract Factory: Amaç, Örnek ve Sorun
- 3. Örnek Çözüm: WidgetFactory ve Güncel UML Gösterimi
- 4. Switch-Case Seçeneği ve Zayıf Yönleri
- 5. Kalıbın Genel Yapısı ve Uygulanabileceği Durumlar
- 6. Uygulama Örneği: Üç Boyutlu Oyun Motoru
- 7. Çözüm 1: Switch-Case Kullanımı
- 8. Çözüm 2: Kalıtımın Doğru Kullanımı
- 9. Kalıtım Yanlış Kurulursa
- 10. Çözüm 3: Abstract Factory Kalıbının Kullanımı
- 11. Çıkarılan Dersler ve Genel Tasarım Kuralları
- Örnek Sorular

1. Kısa Tekrar ve Dersin Düzeni

Hoca derse geçen haftayı özetleyerek başladı. Tasarım kalıpları, nesneye yönelik modellemede sık karşılaşılan problemleri çözmek için önerilen güzel çözüm yollarıdır. İşin nasıl doğru ve nasıl iyi yapılması gerektiğini anlatırlar. Geçen hafta tasarım kalitesinin soyut ve somut ölçütleri tartışılmış, ilk kalıp olarak Model-View-Controller anlatılmıştı.

Dersin yaklaşık sekiz haftası tasarım kalıplarına, son haftaları ise kod kusurları (code smells) ve refactoring konusuna ayrılır. Kalıplar Dörtlü Çete'nin (Gang of Four) 1994 tarihli *Design Patterns* kitabında tanımlanmıştır; derste bunların hepsi (Interpreter dışında) işlenecektir. Code smell'leri literatürde tanımlayan kitap Martin Fowler'ın *Refactoring* kitabıdır; derste bunların yaklaşık yarısı görülecek, geri kalanı için kitap ve Fowler'ın web sitesi önerildi. Hoca, iş hayatında bir kalıbın çözdüğü sorunu farklı ya da benzer biçimde çözmüş olanların deneyimlerini sınıfla paylaşmalarını istedi.

Nesneye yönelik programlamanın temellerinde eksiği olanlara hocanın lisansta verdiği Nesneye Yönelik Programlama dersinin notları, UML sınıf şemalarında eksiği olanlara ise Martin Fowler'ın *UML Distilled* kitabı önerildi. Hocaya göre bu kitap yalnız UML'i anlatmakla kalmaz, her UML öğesinin nesneye yönelimdeki doğru kullanımını, anlamını ve püf noktalarını da anlatır.

Kalıplar kitaptaki sırayla anlatılır. GoF kitabı önce MVC'yi verir, ardından yaratımsal (creational) kalıplarla devam eder. Kalıplar üç kümeye ayrılır: yaratımsal, yapısal (structural) ve

davranışsal (behavioral). Kalıpların adları Türkçeye çevrilmedi, çünkü literatürde yerleşmiş adlardır; hoca bunları orijinal adlarıyla hatırlamamızı istedi.

Hocanın vurgusu: Kalıp adlarını orijinal İngilizce adlarıyla bilmeniz bekleniyor.

Hoca ayrıca tasarım kalıplarının bir yararını da hatırlattı: tecrübenin, deneyimin kalıplar halinde yazılı olarak aktarılabilmesini sağlarlar. Bu yüzden bu konular kendi kendine çalışıp gelişmeye çok uygundur.

2. Abstract Factory: Amaç, Örnek ve Sorun

İlk yaratımsal kalıp Abstract Factory'dir. Slayt (ekrandan, 00:16:50) kalıbı şöyle tanıtır:

- Amaç: Birbirleri ile ilişkili ya da aynı aileden olan nesneleri, sınıflarını belirtmeden oluşturabilmek.
- Örnek: Java'da çeşitli GUI bileşenlerinin görünüşünün (look-and-feel) çalışma anında değiştirilebilmesi.
- Sorun: Taşınabilirliği sağlamak için, istekçinin hangi tür görünüşün hangi sınıftan nesnelerle gerçeklendiğini bilmemesi gerekir.

Amaçtaki "sınıflarını belirtmeden" ifadesi ilk bakışta tuhaf gelebilir: bir nesne oluşturmak için kurucu (constructor) metodu çağırmak gerekir ve kurucunun adı sınıfın adıdır. Bunu mümkün kılan kalıtım ve çok biçimliliktir. Hoca, deneyimli tasarımcıların kalıtımın gücünün yanında tehlikelerinin ve yanlış kullanımlarının da farkında olduğunu, tasarım kalıplarının da doğru modelleme yapmaya yaradığını söyledi.

Her kalıbın tanımında çözmeye çalıştığı problem bir örnek üzerinden verilir. GoF kitabı yayımlandığında Java'nın 1.0 sürümü bile çıkmamıştı. Kitaptaki örnek bu yüzden pencere yöneticileridir: farklı pencere yöneticisi sistemlerinde aynı GUI bileşenleri farklı kodlanır. Kitapta Motif ve Presentation Manager geçer; güncel karşılıkları olarak GNOME ve KDE düşünülebilir.

Amaç taşınabilirliği en üst düzeye çıkarmaktır: yazdığımız kod farklı sistemlerde mümkün olduğunca az değişiklikle çalışmalıdır. Bunun için istekçi (client) yalnız "bana bir pencere çiz, bir kaydırma çubuğu (scroll bar) çiz, bir radyo düğmesi çiz" diyebilmeli; o anda görünüşün Swing'in Metal'i mi, klasik mi, Motif mi olduğu onun için önemli olmamalıdır. Client bu işi somut gerçeklemelere bağımlı olmadan yapabilmelidir.

3. Örnek Çözüm: WidgetFactory ve Güncel UML Gösterimi

3.1 Kitaptaki çözüm

Widget, her GUI bileşenine (düğme, pencere, kaydırma çubuğu, radyo düğmesi vb.) verilen ortak addır. Client bir widget fabrikasıyla (`WidgetFactory`) konuşur ve örneğin onun `CreateScrollBar()` metodunu çağırarak "bana bir kaydırma çubuğu ver" der. Geriye `ScrollBar` türünden bir nesne döner. `WidgetFactory` soyut bir sınıftır. O anda Motif pencere yöneticisi çalışıyorsa client'a aslında somut `MotifWidgetFactory` alt sınıfından bir nesne verilmiştir. Client onu kalıtımın yerine geçirebilirlik ilkesi nedeniyle yalnız bir `WidgetFactory` olarak görür; ondan aldığı kaydırma çubuğu da arka planda bir `MotifScrollBar`'dır.

Bu esnekliği sağlayan, kalıtımın doğru kullanılması ve çok biçimliliktir: alt sınıftan bir nesne, üst sınıftan bir nesnenin yerine her zaman ve her bağlamda kullanılabilir.

Slayttaki şema (ekrandan, 00:20:00) GoF kitabından alınmıştır:

```
Client ──> WidgetFactory {CreateScrollBar(), CreateWindow()}   (italik = soyut)
Client ──> Window      Window    △── PMWindow, MotifWindow
Client ──> ScrollBar   ScrollBar △── PMScrollBar, MotifScrollBar
WidgetFactory △── MotifWidgetFactory {CreateScrollBar(), CreateWindow()}
              △── PMWidgetFactory    {CreateScrollBar(), CreateWindow()}
MotifWidgetFactory ··> MotifWindow, MotifScrollBar
PMWidgetFactory    ··> PMWindow, PMScrollBar
```

3.2 Şemanın güncel UML'e çevrilmesi

Hoca bu şemayı GoF kitabından doğrudan ekran görüntüsü alarak ders notlarına koyduğunu, ama kitap yazıldığında UML sınıf şemalarının henüz standartlaşmadığını söyledi. Bu yüzden kitaptaki kalıtım ve ilişki okları güncel standarttan farklıdır; hoca kitaptaki oku ER şemalarındaki "karga ayağı" gösterimine benzetti. Ardından şemayı Violet UML Editor ile güncel standarda göre yeniden çizdi.

> **Ek bilgi (derste anlatılmadı):** GoF kitabındaki sınıf şemaları büyük olasılıkla Rumbaugh'un OMT gösterimiyle çizilmiştir. UML, OMT ile Booch ve Jacobson'ın yöntemlerinin birleşmesinden doğmuş ve büyük olasılıkla 1997'de OMG standardı olmuştur. Bu nedenle kitaptaki içi dolu ok uçları ve üçgenler bugünkü UML'deki anlamlarıyla birebir örtüşmez.

Hoca çizim sırasında şu UML ayrıntılarını hatırlattı:

- Soyut sınıflar ve metotlar kitapta italik yazılmıştır. Elle çizimde italik yazıyı ayırt etmek zordur; hoca kendi el yazısının kötü olduğunu, okuyanın harfleri bile seçemeyebileceğini söyleyerek soyut sınıfları «`abstract`» stereotipiyle göstermeyi tercih ettiğini söyledi. Violet'te bu stereotip doğrudan verilemediği için bir interface öğesi oluşturup "interface" yazısını "abstract" olarak değiştiriyor.
- Üye alanlar orta bölmeye, üye metotlar alt bölmeye yazılır. GoF bu şemada üye alan göstermemiştir. Bütün metotları soyut olan ve üye alanı olmayan bir sınıf rahatlıkla interface olarak da kodlanabilir; kitap yazıldığında interface kavramını her dil desteklemediği için kitap buna pek değinmez.
- Public metot `+`, private `-` ile gösterilir. Metotlar genelde public tanımlanır; bir nesnenin iç çalışmasına ilişkin ve başka sınıflarca bilinmemesi gereken ayrıntıları içeren metotlar private olabilir. Üye alanlar veri gizleme ilkesine uymak için her zaman private tanımlanır.
- Dönüş türleri yazılmalıdır: `createScrollBar()` bir `ScrollBar`, `createWindow()` bir `Window` döndürür.
- Soyut metotlar `{abstract}` ile gösterilir. Buradaki küme parantezleri metot gövdesi değildir; `{final}` ya da `{abstract, final}` gibi nitelikler de aynı biçimde yazılır. Alt sınıflardaki metotlar somuttur.
- Somut fabrika ile ürettiği ürünler arasında, üye alan olmadan yalnız dönüş değeri, metot parametresi ya da geçici değişken olarak kullanma (bağımlılık) ilişkisi vardır; bu kesikli okla gösterilir.

Güncel UML ile çizilmiş şema (ekrandan, 00:39:30):

```
«abstract» WidgetFactory
  + createScrollBar() : ScrollBar {abstract}
  + createWindow() : Window {abstract}
    △── MotifWidgetFactory
    │      + createScrollBar() : MotifScrollBar
    │      + createWindow() : MotifWindow
    △── PMWidgetFactory
           + createScrollBar() : PMScrollBar
           + createWindow() : PMWindow

«abstract» Window      △── MotifWindow, PMWindow
«abstract» ScrollBar   △── MotifScrollBar, PMScrollBar

Client ··> WidgetFactory, Window, ScrollBar          (kesikli ok: kullanma)
MotifWidgetFactory ··> MotifWindow, MotifScrollBar
PMWidgetFactory    ··> PMWindow, PMScrollBar
```

> **Ders sonrası not:** Hocanın PNG olarak dışa aktardığı şemada `Window` altındaki iki sınıf yanlışlıkla `MotifScrollBar` ve `PMScrollBar` adını taşıyor (ekrandan, 00:39:30). Hoca bunu fark edip bu sınıfları Window olarak düzeltti (00:45:52). Yukarıdaki şemada doğru adlar (`MotifWindow`, `PMWindow`) kullanılmıştır.

> **Hocanın vurgusu:** Ders kitabında ve ders notlarında GoF'un eski gösterimini görseniz de sınavlarda güncel UML standardına uygun çizim yapın. Hoca bu şemaları tek tek güncellemediğini, ama doğrusunun bu olduğunu açıkça belirtti.

3.3 Violet UML Editor üzerine

Hoca basit UML şemaları için Violet UML Editor kullanıyor. Bu basit bir jar dosyasıdır; sistemde JRE (ya da JRE'yi içeren JDK) kuruluysa çift tıklayarak çalışır. Şemalar kaydedilip açılabilir (dosya `.class.violet` uzantısıyla kaydedilir) ve "Export to Image File" ile aynı adla PNG olarak dışa aktarılabilir. Kod üretme ya da koddan şema çıkarma yeteneği yoktur. Hoca bir dönem koddan şema üretmek için Eclipse'in eUML2 eklentisini kullanmış, ama ürettiği şemalar karmaşık olduğu için vazgeçmiş; ilerideki bazı slaytlarda bu eklentiyle üretilmiş ve sadeleştirilmiş şemalar görülecek.

Violet'in hocanın kullandığı sürümünde bazı kesikli çizgiler düz çizgiye dönüşebiliyor ve dirsekli çizgiler pek desteklenmiyor. Yeni sürümü çıkmış ama dosya biçimi eski dosyalarla uyumsuz olduğu için hoca eski sürümü kullanıyor.

4. Switch-Case Seçeneği ve Zayıf Yönleri

Kalıbın önerdiği çözüm yerine akla ilk gelen bir tasarım da uygulanabilir. Slayt (ekrandan, 00:40:00) bunu şöyle verir:

- Diğer seçenekler:
  - Switch – Case:
    - `WidgetFactory.setLoookAndFeel( val: EnumLookFeel )`
    - `switch( val ) { … }`
    - Zayıf yönü: Yeni görünüş biçimleri için yeni enum değerleri ve switch içerisine yeni case komutları eklemek gerek.
  - Başka?

Hoca burada önemli bir ipucu verdi:

> **Hocanın vurgusu:** Nesneye yönelik kod yazarken kendinizi switch-case kullanırken görürseniz, büyük olasılıkla doğru bir tasarım yapmamışsınızdır. Bu, kalıtım ilişkisini ve onun getirdiği çok biçimliliği kullanmanız gerektiğinin çok kuvvetli bir işaretidir. Bir yerde switch-case görürseniz bu bir alarm olmalıdır.

Switch-case çözümünün iki zayıf yönü vardır. Örnekte iki pencere yöneticisi (Motif ve Presentation Manager) vardır. GNOME ve KDE'yi de desteklemek gerektiğinde, enum'a yeni değerler eklenir ve "eğer GNOME ise şunu yap, KDE ise bunu yap" diye iki yeni case yazılır. İlk iki case'te 20-30 satır kod varsa, aynı uzunlukta iki kod bloğu daha eklenir:

- Bakım zorlaşır: yeni görünüş biçimleri geldikçe kod aşırı uzar ve daha fazla angarya iş yapılır.
- Uzun metot oluşur: ileride kod kusurları arasında görülecek "Long Method" kusuru. Uzun metotların okunabilirliği ve anlaşılabilirliği çok düşüktür.

Hoca, geriye doğru uyumluluk gerekse bile yeni bir kalıtım hiyerarşisi kurulabileceğini ekledi.

5. Kalıbın Genel Yapısı ve Uygulanabileceği Durumlar

5.1 Kalıp yapısı

Bu çözüm yalnız pencere yöneticisi ya da look-and-feel için değil, benzer başka amaçlar için de kullanılabildiği için genel bir kalıp yapısı olarak ifade edilir. Kalıbın odağındaki sınıfın adı `AbstractFactory` olduğu için kalıbın adı da Abstract Factory'dir.

> **Hocanın vurgusu:** Bir kalıbın, sınıfın ya da metodun adında "factory" kelimesini görürseniz, onun bir nesne oluşturmakla ilgili bir görevi vardır.

Kalıp yapısı (ekrandan, 00:45:00):

```
Client ──> AbstractFactory {CreateProductA(), CreateProductB()}
Client ──> AbstractProductA      AbstractProductA △── ProductA2, ProductA1
Client ──> AbstractProductB      AbstractProductB △── ProductB2, ProductB1
AbstractFactory △── ConcreteFactory1 {CreateProductA(), CreateProductB()}
                △── ConcreteFactory2 {CreateProductA(), CreateProductB()}
```

```
ConcreteFactory1 ··> ProductA1, ProductB1
ConcreteFactory2 ··> ProductA2, ProductB2
```

Kalıp bileşenleri (aynı slayt):

- ProductXY: Oluşturulacak nesneler, yani ürünler. X (A, B, …) hangi ürün olduğunu söyler: örneğin pencere ya da kaydırma çubuğu. Y (1, 2, …) bu ürünün hangi aileden olduğunu söyler: örneğin Motif ailesinin penceresi ya da Presentation Manager ailesinin penceresi.
- AbstractProductX: X ürünlerinin arayüzlerini tanımlar.
- ConcreteFactoryY: Y ürünlerini oluşturur. Y ailesi GNOME, KDE ya da Presentation Manager olabilir.
- AbstractFactory: Ürün oluşturmak için gerekli komutların imzalarını içerir. Soyut metodun yalnız imzası tanımlanır, gövdesi somut alt sınıflarda kodlanır.

Özetle: soyut bir fabrikamız ve soyut ürünlerimiz vardır; client yalnız bunlar üzerinden çalışır. Ona somut bir fabrika verirsek arka planda o fabrikanın ürettiği ürünlerle çalışır, ama client'ın bundan haberi bile olmaz.

5.2 Uygulanabileceği durumlar

Hoca her kalıptan sonra uygulanabileceği alanların verildiğini, ama bunları okumanın tek başına çok anlamlı olmadığını söyledi. Bir problemle karşılaşınca "hangi kalıp uygun?" sorusuna, kalıpların uygulanabileceği anlar tablosuna bakarak çok ilerlenemez. Asıl yol kalıpların genel yapılarını hatırlamak, kitaplardaki ve ders notlarındaki örnekleri incelemek ve yeni bir problemin daha önce görülenlere ne oranda benzediğini fark etmektir. Bu yetenek ancak deneyimle kazanılır ve keskinleşir.

Slayttaki maddeler (ekrandan, 00:50:00):

- Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar. Client ortak soyut sınıfın soyut metotları üzerinden çağrı yapar; somut fabrikanın hangi türden nesneyi hangi adımlarla oluşturduğunu bilmek zorunda kalmaz.
- Bir sistemin farklı ailelerden herhangi birindeki nesneler tarafından yapılandırılması gerektiğinde. Örneğin "bir düğme istiyorum; şu an hangi pencere yöneticisi kullanılıyorsa o türden olsun".
- İlişkili ürün ailelerinden oluşan nesnelerin birlikte kullanılması gerektiğinde.
- Bir ürün ailesindeki nesnelerin bir sınıf kütüphanesi (ya da framework) olarak sunulacağı, ancak gerçeklemeleri yerine yalnızca arayüzlerinin açığa çıkarılacağı durumlarda.

Son madde için interface gerçeklemesi ya da ileride görülecek Facade kalıbı da benzer iş görür; ama Facade'ın çok daha fazla karmaşıklığı olduğu için bu tür problemlerde Abstract Factory'yi tercih etmek daha yerindedir.

6. Uygulama Örneği: Üç Boyutlu Oyun Motoru

6.1 Örneklerin anlatım biçimi

Hoca ders kapsamında her kalıp için (bir iki çok basit kalıp dışında) GoF'un anlattığından ayrı bir örnek hazırladığını söyledi. Bu örnekleri ya ilk derste verilen, kalıpları örneklerle anlatan kaynak kitaplardan seçmiş ya da kendisi oluşturmuştur. Kendi örneğini özellikle şu durumlarda oluşturmuş: bazı kitaplar bir örnekte birden fazla kalıbı birlikte kullanır; örneğin Abstract Factory'nin örneği daha sonra görülecek bir kalıpla birleştirilerek anlatılmışsa, henüz görülmemiş bir kalıbı kullanmak eğitim açısından uygun olmaz. Bazen de uygun örnek bulamamıştır. Slaytın altında kaynak yazar; kaynak "YES" ise örnek hocanın kendisine aittir (Yunus Emre Selçuk'un baş harfleri).

Örneklerin anlatım biçimi şöyledir:

1. Önce akla ilk gelen, belki de doğru olmayan çözümle modellenir.
2. Sonra gerçekleştirmesi epey angarya olan bir değişiklik isteği eklenir.
3. Aynı problem kalıpla çözülür ve aynı değişikliğin kalıplı tasarımda ne kadar rahat yapılabildiği deneyimlenir.

Böylece kalıpların yararları somut olarak görülür. Hoca bu anlatım tarzını bazı kitaplarda görüp beğendiğini söyledi.

Örnek kodlar en azından derleme hatası vermeyen ve amaca ana hatlarıyla nasıl hizmet ettiğini gösteren iskelet kodlardır. Tüm kaynak kodlar zip dosyası olarak ders notlarıyla birlikte hocanın web sayfasındadır.

> **Hocanın vurgusu:** Sınavlarda kod yazmanız istenmez, uygun bir tasarım yapmanız istenir. Kendinizi ifade etmekte zorlandığınız yerlerde küçük kod parçaları yazabilirsiniz.

6.2 Problem

Bir üç boyutlu oyun motoru hazırlıyoruz ve mümkün olduğunca geniş bir donanım aralığına hitap etmek istiyoruz: farklı işletim sistemleri ve farklı ekran kartları desteklenmeli. Bu nedenle oyun motoru OpenGL kütüphanesinin ya da DirectX kütüphanesinin değişik sürümleriyle çalışabilmelidir. Bir sistemde yalnız OpenGL sürücüleri, bir başkasında yalnız DirectX destekli sürücüler olabilir.

Motor bu kütüphanelerin birçok bileşenini kullanır:

- Renderer: üç boyutlu noktalardan oluşan görüntüyü, her harekette yeni bakış açısını hesaplayarak ekrana çizer.
- Shader: ışık kaynaklarının konumuna göre gölgeleri çizer, oluşturur.

Bu bileşenler ya OpenGL'den ya da DirectX'ten gelecektir. Problem bunun nasıl tasarlanacağıdır.

7. Çözüm 1: Switch-Case Kullanımı

7.1 Kod ve sınıf şeması

İlk akla gelen çözüm switch-case'tir (ekrandan, 01:10:00). `ApControl` (Application Control) oyunun ana modülüdür:

```
package dp.abstractFactory.solution1;
public class ApControl {
   private LibraryType library;
   private OpenGLRenderer or;
   private OpenGLShader os;
   private DirectXRenderer dr;
   private DirectXShader ds;
   public ApControl( ) { /*library nesnesini ilklendirerek kütüphaneyi belirle*/ }
   void doRendering ( ) {
      switch (library){
         case OpenGL:
               or.renderOpA(); or.renderOpB(); or.renderOpC(); break;
         case DirectX:
               dr.renderOpA(); dr.renderOpB(); dr.renderOpC(); break;
      }
   }
   void doShading ( ) {
      switch (library){
         case OpenGL:
               os.shadeOpA(); os.shadeOpB(); os.shadeOpC(); break;
         case DirectX:
               ds.shadeOpA(); ds.shadeOpB(); ds.shadeOpC(); break;
      }
   }
}
```

`LibraryType`, `OpenGL` ve `DirectX` değerlerini içeren bir enum'dur. Kurucu, `library` nesnesini ilklendirerek o anda hangi kütüphaneyle çalışılacağını belirler. Render gerektiğinde sistem

OpenGL'i destekliyorsa `or` nesnesi, DirectX'i destekliyorsa `dr` nesnesi üzerinden işlem yapılır; gölgelendirme de aynı mantıkla yürür.

`OpenGLRenderer` gibi sınıfların `renderOpA()`, `renderOpB()`, `renderOpC()` metotları iskelet kodda boştur (`//do rendering in DirectX` yorumuyla, ekrandan 01:30:00), ama gerçekte ilgili komutlarla dolu, karmaşık metotlar olacaktır. Hoca üç boyutlu grafiğin en karmaşık problemlerden biri olduğunu hatırlattı.

Sınıf şeması (ekrandan, 01:15:00):

```
ApControl ──> OpenGLShader     {+ shadeOpA(), + shadeOpB(), + shadeOpC()}
ApControl ──> DirectXShader    {+ shadeOpA(), + shadeOpB(), + shadeOpC()}
ApControl ──> OpenGLRenderer   {+ renderOpA(), + renderOpB(), + renderOpC()}
ApControl ──> DirectXRenderer  {+ renderOpA(), + renderOpB(), + renderOpC()}
```

`ApControl` bu dört sınıfla bire bir sahiplik ilişkisi içindedir (her birinden tek bir üye alan). Hoca UML şemasında her ayrıntının verilmek zorunda olmadığını, bire bir sahiplikte ok üzerindeki niceliği genelde yazmadığını, ama bire çok sahiplikte (örneğin bir liste ya da dizi) * koyacağını söyledi. Ancak sınavlarda kod istenmediği için şemanın olabildiğince ayrıntılı çizilmesi, öğrencinin konuyu doğru anladığının görülmesine yardım eder.

> **Hocanın vurgusu:** UML'in bu temellerini bilmeniz bekleniyor.

Şemaya bakınca belirgin bir terslik görünmez, ama hocayı switch-case kullanımı son derece rahatsız eder. Gerçek bir oyunda `doRendering` ve `doShading`'in yanında `doNotation`, `doCollisionDetection` gibi birçok metot olacak, her biri desteklenen kütüphane sayısı kadar case içerecek; kod upuzun ve karmaşık olacaktır.

7.2 Değişiklik isteği: DirectX'in yeni sürümü

Şimdi DirectX'in yeni bir sürümü geldi (hoca önce 11, sonra 12 dedi): 10'u da 12'yi de desteklemek istiyoruz. DirectX'in her sürümünün kütüphaneleri birbirinden farklıdır. Çözüm 1'de yapılması gerekenler (ekrandan, 01:20:00 ve 01:30:00):

1. `LibraryType` enum'una yeni bir değer eklenir.
2. İki yeni sınıf oluşturulur: `DirectXRenderer12` ve `DirectXShader12`. Bu sınıfların metotları her durumda DirectX 12'nin getirdiği farklılıklara göre kodlanmalıdır; bundan hiçbir tasarımda kaçış yoktur.
3. `ApControl`'e yeni üye alanlar (`DR12`, `DS12`) eklenir.
4. `doRendering` ve `doShading` metotlarının her birine yeni bir case eklenir.

Hoca kopyala-yapıştır sırasında yeni nesneleri yeniden adlandırmayı unutmanın bu tür programlamanın tipik tuzağı olduğunu gösterdi. Sonuçta yeni bir DirectX sürümü için, yeteneği sayısınca (burada iki) yeni sınıf oluşturuldu ve iki sınıfın koduna (enum ve `ApControl`) müdahale edildi.

Bir de gerçek zamanlı texture mapping desteklenecek olsun: her kütüphane ve her sürüm için ayrı bir texture mapping sınıfı ve `ApControl`'de yeni case'ler gerekir. Her yeni özellik ya da sürüm için birçok yere dokunmak gerekir.

Satır sayısı (Line of Code) iki tasarım arasında belki çok farklı olmayacaktır. Ama mesele satır sayısı değil, bakım ya da yenilik çalışmasında gereken çabayı aşırı artırmamaktır.

> **Hocanın vurgusu:** Kendinizi switch-case kullanırken görüyorsanız büyük olasılıkla çok biçimliliğe ihtiyacınız vardır. Daha küçük bir olasılıkla da sorumlulukları doğru sınıfların doğru metotlarına atamamışsınızdır.

8. Çözüm 2: Kalıtımın Doğru Kullanımı

Abstract Factory kalıbını henüz bilmediğimizi varsayalım. Nesneye yönelik bir tasarımcı olarak ilk tasarımın sakıncasını gördük; doğru bir kalıtım ilişkisi kurabilir miyiz?

Bunun için şu ilke kullanılır: **değişenin ne olduğunu bul ve onu kalıtım hiyerarşisine koy.** Her durumda rendering ve shading işlemleri yapılacaktır; değişen, bu işlemlerin OpenGL'de

ve DirectX'te farklı yapılmasıdır. Değişeni doğru tanımlamak önemlidir, çünkü kalıtım hiyerarşisi çok kolay ters kurulabilir. `ApControl`, arka planda OpenGL mi, DirectX 10, 11 ya da 12 mi kullanıldığından bağımsız çalışabilme lüksüne sahip olmalıdır. Bu yüzden soyut bir `Shader` ve soyut bir `Renderer` sınıfı, bunların altında da somut `OpenGLShader`, `DirectXShader`, `OpenGLRenderer` ve `DirectXRenderer` sınıfları kurulur.

Kod ve şema (ekrandan, 01:27:45):

```
package dp.abstractFactory.solution2;
public class ApControl {
   private LibraryType library;
   private Renderer r;
   private Shader s;
   //library nesnesini ilklendirip kütüphane belirle
   public ApControl( ) {
      switch (library){
         case OpenGL:
            r = new OpenGLRenderer();
            s = new OpenGLShader();
            break;
         case DirectX:
            r = new DirectXRenderer();
            s = new DirectXShader();
            break;
      }
   }
   void doRendering ( ) {
      r.renderOpA(); r.renderOpB(); r.renderOpC();
   }
   void doShading ( ) {
      s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
   }
}
```

```
ApControl ──> «Abstract» Shader
                + void shadeOpA() {abstract}
                + void shadeOpB() {abstract}
                + void shadeOpC() {abstract}
                  △── OpenGLShader, DirectXShader
ApControl ──> «Abstract» Renderer
                + void renderOpA() {abstract}
                + void renderOpB() {abstract}
                + void renderOpC() {abstract}
                  △── OpenGLRenderer, DirectXRenderer
```

Hangi kütüphaneyle çalışıldığını tutmak hâlâ gerekli olduğundan enum kalır, ama artık üye alan olarak yalnız bir renderer ve bir shader vardır. Kurucu biraz karmaşıklaştı, çünkü switch-case artık oradadır: donanım OpenGL olarak algılanmışsa OpenGL nesneleri, DirectX ise DirectX nesneleri ilklendirilip `r` ve `s`'ye atanır. Render ve shade işlemleri doğrudan `r` ve `s` üzerinden yapılır. Önceden switch-case iki yerdeydi (hem rendering hem shading içinde); şimdi tek yerde, kurucudadır.

DirectX 12 eklemek için (ekrandan, 02:20:00) enum'a bir değer eklenir, `DirectX12Renderer` ve `DirectX12Shader` kardeş sınıfları oluşturulup kodlanır ve `ApControl`'ün kurucusuna tek bir case eklenir. Sınıf sayısı olarak yine dört yerde değişiklik olur, ama `ApControl`'deki değişiklik kurucudaki basit bir case'tir. Çözüm 1'de değişiklikler karmaşık render ve shade işlemlerinin içindeydi.

Peki karmaşıklık nereye gitti? Kaybolmadı; `DirectXRenderer` ve `DirectXShader` gibi sınıfların içine kodlandı. Yeni bir sistem desteği için her türlü uzun ve karmaşık işlemin içindeki switch'e n tane case eklemek yerine, kurucudaki kısacık switch'e bir case eklemek yetti. Geri kalan her şeyi kalıtım ve çok biçimliliğin gücü sağladı.

9. Kalıtım Yanlış Kurulursa

9.1 Kütüphane başına sınıf ve delegate metotlar

Hoca, kalıtım hiyerarşisi yanlış kurulursa ne olacağını göstermek için Violet'te bir alternatif çizdi. Bu kez üst sınıflar `OpenGL` ve `DirectX` olsun (ekrandan, 01:40:00 ve 01:50:00):

```
ApControl ──> OpenGL
                - OpenGLRenderer r
                - OpenGLShader s
                + void shadeOpA() ... shadeOpC()
                + void renderOpA() ... renderOpC()
              OpenGL ──> OpenGLRenderer, OpenGLShader
ApControl ──> DirectX
                - DirectXRenderer r
                - DirectXShader s
                + aynı altı metot
              DirectX ──> DirectXRenderer, DirectXShader
(DirectX 12 için) DirectX12 {- DirectX12Renderer r, - DirectX12Shader s, aynı altı metot}
              DirectX12 ──> DirectX12Renderer, DirectX12Shader

Not: public void shadeOpA() { s.shadeOpA(); //delegation }
```

Hoca çizerken burada aslında kalıtımın söz konusu olmadığını, `OpenGL` sınıfının renderer ve shader'a sahip olduğu için bir sahiplik ilişkisi kurulduğunu fark etti. `ApControl` işini doğrudan `OpenGLRenderer` üzerinden değil, `OpenGL` sınıfı üzerinden yapar. `OpenGL` sınıfındaki `shadeOpA()` metodu yalnız `OpenGLShader`'daki `shadeOpA()`'yı çağırır; yani bir köprü, iletici görevi görür. Bu bir tür proxy'dir; genel bilgisayar mühendisliği anlamında delegate (yetki devri) de denir: metot sorumluluğu başkasına aktarır.

Hocanın delegation benzetmesi: bir öğrenci ders tanıtım formu ya da staj defteri imzası için bölüm başkanına gider. Bölüm başkanı toplantıda olduğu için öğrenciyi, bu işi yapmaya yetkili yardımcısına yönlendirir. İşi kendisi yapmaz, doğrudan başkasına iletir.

Bu tasarımda bütün render ve shade metotları her kütüphane sınıfında delegate metot olarak tekrar yazılmalıdır. Hoca şemaya "delegate metotlar" ve "Tercih edilmeyecek modelleme" notlarını ekledi. Rahatsız eden iki nokta:

- Delegate metotların fazlalığı.
- DirectX 12 geldiğinde iki değil üç sınıf eklemek gerekir: `DirectX12`, `DirectX12Renderer` ve `DirectX12Shader`. Düzgün kalıtımla yalnız iki kardeş sınıf yeterlidir.

9.2 Araya soyut bir GameEngine koymak

Delegate metot fazlalığını azaltmak için hoca ayrı bir şemada araya soyut bir `GameEngine` sınıfı koydu: bütün render ve shade metotları onda soyut olarak tanımlanır, `ApControl` doğrudan `OpenGL` ya da `DirectX` ile değil `GameEngine` ile ilişki kurar, `OpenGL`, `DirectX` ve `DirectX12` de onun kardeş alt sınıfları olur. Böylece `ApControl` ile oyun motoru kodu arasına bir katman konmuş olur. Ama arka planda karmaşıklık sürer: yeni bir DirectX sürümü için yine üç sınıf eklemek gerekir. Bu çözüm de tercih edilmemesi gereken bir çözüm olma özelliğini korur ve switch-case'li ilk çözüme göre bir avantaj getirmez.

Hoca bu girişimin ders saatini kaybetmek olmadığını, hatalardan öğrenmek için yapıldığını söyledi: switch-case'ten kurtulmak için kalıtım kullanıldı, ama hiyerarşi yanlış kurulduğu için buraya gelindi.

9.3 Oyunun tamamını türetmek

Dersin sonuna doğru hoca aklına gelen bir kötü seçeneği daha çizdi (ekrandan, 02:39:20): kütüphaneleri değil, oyunun tamamını türetmek.

```
«Abstract» APControl
   △── OpenGL_APControl  ──> OpenGLShader, OpenGLRenderer
   △── DirectX_APControl ──> DirectXShader, DirectXRenderer
```

Bu durumda her alt sınıf kendi renderer ve shader'ına bire bir sahip olur. Kod uzunluğu açısından çok kötü değildir, bir tür kabul edilebilir çözüm gibi görünür. Ancak şu sorun vardır: oyunun mantığında rendering ve shading işlemlerini etkileyen bir hata varsa (hata OpenGL ya da DirectX kütüphanesinde değil, oyun programının onları kullanma biçimindedir), bu hata hem `OpenGL_APControl`'de hem `DirectX_APControl`'de, DirectX 12 de eklenirse üç ayrı yerde düzeltilmek zorunda kalınır. Aynı ya da benzer kodun birden fazla yerde tekrarlanması bir tasarım kusurudur. Bu yüzden bu çözüm de tercih edilmez; burada kalıtım yanlış amaçla kullanılmıştır.

10. Çözüm 3: Abstract Factory Kalıbının Kullanımı

10.1 Kalıbın örneğe uyarlanması

Kalıp yapısı hatırlanırsa: `ApControl` client'tır. Renderer ve shader ürünlerine ihtiyaç vardır; bunlar OpenGL ailesinin ya da DirectX ailesinin üyesi olabilir. Bu adlandırmayla kalıp doğrudan uyar (ekrandan, 02:05:45):

```
ApControl ──> «Abstract» ComponentFactory
                + createRenderer() : Renderer {abstract}
                + createShader() : Shader {abstract}
                  △── OpenGLFactory
                  │     + createRenderer() : OpenGLRenderer
                  │     + createShader() : OpenGLShader
                  △── DirectXFactory
                        + createRenderer() : DirectXRenderer
                        + createShader() : DirectXShader
ApControl ──> «Abstract» Shader    △── OpenGLShader, DirectXShader
ApControl ──> «Abstract» Renderer  △── OpenGLRenderer, DirectXRenderer
```

Slayttaki açıklama: "Ana program kurucusunda uygun bir ComponentFactory gerçeklemesi ilklendirilir." `ComponentFactory` soyut fabrikadır; `createRenderer()` ve `createShader()` metotları soyut `Renderer` ve `Shader` döndürür. Bunlar ailelerin üyelerinin türlerini simgeleyen soyut sınıflardır; arka planda soyut işaretçiye ya bir OpenGL ya da bir DirectX nesnesi verilir. OpenGL ile çalışılacaksa `OpenGLFactory`, DirectX ile çalışılacaksa `DirectXFactory` kullanılır. `OpenGLFactory`'nin `createRenderer()` metodu bir `OpenGLRenderer`, `createShader()` metodu bir `OpenGLShader` döndürür; `DirectXFactory` de benzer kodlanır.

Kod (ekrandan, 02:07:40):

```
package dp.abstractFactory.solution3;
public class ApControl {
   private ComponentFactory library;
   private Renderer r;
   private Shader s;
   public ApControl( ) {
      /*library nesnesini ilklendirerek
        kütüphane türünü belirle
        library = new OpenGLFactory();
        library = new DirectXFactory();
       */
      r = library.createRenderer();
      s = library.createShader();
   }
   void doRendering ( ) {
      r.renderOpA(); r.renderOpB(); r.renderOpC();
   }
   void doShading ( ) {
      s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
   }
}
```

10.2 Fabrika nereden seçilecek?

Kalıplı çözümde de en başta DirectX sisteminde mi OpenGL sisteminde mi çalışıldığına karar verilmesi gerekir. Hoca slayttaki kodda bu işi "halının altına süpürdüğünü" söyledi: `library` üye alanı burada ilklendirilmemiştir. `library` henüz `null` olduğu için bu haliyle `library.createRenderer()` çağrısı çalışma anında `NullPointerException` verir. Bu yüzden derste kodun yorum kısmına üç seçenek eklendi (02:22:42):

- A. `library` nesnesini ilklendirip kütüphane türünü switch-case ile belirlemek (önceki örnekteki gibi).
- B. `library` nesnesini kurucu metoda parametre olarak vermek.
- C. Farklı kütüphaneler için farklı dağıtımlar yapmak.

Hoca seçenekleri örneklerle açıkladı. Oyunlarda genelde bir Options ekranı bulunur ya da oyun başlamadan önce bir yapılandırma (configurator) uygulaması çalışır; donanımın OpenGL mi DirectX mi desteklediğini bulup bir yapılandırma dosyasına işaret koyar. Oyun bu dosyadaki satırı okur ve "DirectX yazıyorsa DirectX, değilse OpenGL" diye bir switch-case ile seçim yapar. Ya da yazılımın farklı sürümleri dağıtılır: OpenGL müşterileri için `OpenGLFactory`, DirectX müşterileri için `DirectXFactory` yazılıp paketlenen sürümler.

Ara sürümde kurucu şöyleydi (ekrandan, 02:10:00):

```
public ApControl( ComponentFactory library ) {
   this.library = library;
   /*library nesnesini ilklendirerek kütüphane türünü
    * switch-case ile belirle
   library = new OpenGLFactory();
   library = new DirectXFactory();
   */
   r = library.createRenderer();
   s = library.createShader();
}
```

Hoca üç seçenekten parametre olarak vermeyi (B) tercih etti.

> **Hocanın vurgusu:** Tasarım kalıbını kullansak bile switch-case kaybolmaz. Kalıtımın doğru kullanımında olduğu gibi, dağılmış kopyalarını bir yerde toplar; bu yer kurucu olmak zorunda değildir, bir yapılandırma programı da olabilir. Bu düşmanı tamamen öldüremiyoruz, yalnız ilklendirme noktasına kadar geri çekiyoruz.

Bu açıdan kalıbın diğer çözümlere bir avantajı yoktur, çünkü aynı taktik diğer çözümlere de uygulanabilir.

10.3 Çözüm 2 ile Çözüm 3'ün karşılaştırılması

Hoca Eclipse'te iki `ApControl`'ü yan yana karşılaştırdı (Compare With Each Other). Çözüm 3'te kurucu biraz sadeleşti, ama bu sadeleşme şart değildir. Fabrikayı kurucuya parametre olarak vermek, seçim karmaşıklığını koddan çıkarıp yapılandırma dosyasına öteler. Ancak Çözüm 2'de de renderer ve shader nesneleri (ya da `LibraryType` değeri) kurucuya parametre olarak verilebilir; birinde switch-case kullanılırsa öbüründe de kullanılır. Geri kalan kısımlarda fark yoktur. Renderer ve Shader sınıfları iki çözümde aynıdır, yalnız paket adları farklıdır. Çözüm 3'te ek olarak `ComponentFactory`, `OpenGLFactory` ve `DirectXFactory` sınıfları gelmiştir; bu tasarıma biraz ek yük getirir.

DirectX 12 eklemek için Çözüm 3'te (02:17:35) `DirectX12Renderer` ve `DirectX12Shader` kardeş sınıflarının yanında bu kez bir de `DirectX12Factory` eklemek zorunludur. Yani Çözüm 2'ye göre bir sınıf fazladır.

Değişiklik maliyetlerinin dersteki anlatıma göre karşılaştırması:

Tasarım	DirectX 12 için yeni sınıf	Diğer değişiklikler
Çözüm 1 (switch-case)	2 (`DirectXRenderer12`, `DirectXShader12`)	Enum'a değer, `ApControl`'e üye alanlar, her `do...` metoduna case
Çözüm 2 (doğru kalıtım)	2 kardeş sınıf	Enum'a değer, kurucuya bir case
Kütüphane başına sınıf (delegate)	3 (`DirectX12`, `DirectX12Renderer`, `DirectX12Shader`)	Delegate metotların tekrar yazılması
Çözüm 3 (Abstract Factory)	3 (`DirectX12Factory` + 2 ürün sınıfı)	Fabrika seçim noktasına ekleme

Hocanın vardığı sonuç: bu örnekte kalıplı çözümün, kalıtımı düzgün kullanan çözüme göre görünür bir avantajı yoktur. Her zaman illa tasarım kalıbı kullanmak diye bir amaç ya da gerekçe yoktur. Kalıp kullanmadan ama kalıtımı doğru kullanmaya çalışarak yapılan tasarım daha iyi bir yere getirdi. Ancak bu tehlikeli bir yoldur: kalıtım hiyerarşisini kurarken hata yapılsaydı (Bölüm 9'daki gibi), sonunda kalıplı çözümden de kötü bir yere varılacaktı. Kalıtımı doğru kullanmak acemi biri için risklidir, deneyimli biri için bu risk büyük ölçüde ortadan kalkar. Kalıbı kullanmak bir gereksinimse (örneğin bir sınav gereksinimi ya da ders anlatmak için örnek gereksinimi) bu örnek problem Abstract Factory'ye bu şekilde uyarlanabilir.

10.4 Son hal: Preserve Whole Object

Slayttaki ek notlar (ekrandan, 02:30:00):

- Çözümlerdeki soyut sınıflar yerine arayüz kullanımı da mümkündür.
- Tüm çözümlerde library nesnesi `ApControl` kurucusuna parametre olarak verilirse daha doğru bir iş yapılmış olunur. Neden? Bize ne kazandırır?

Hoca dersin sonunda slaytı şöyle güncelledi (ekrandan, 02:31:40): "Refactoring konusunu gördükten sonra, Preserve Whole Object eyleminin yürütülmüş hali tercih edilir: Neden? Bize ne kazandırır?"

```
package dp.abstractFactory.solution3;
public class ApControl {
   private ComponentFactory factory;
   public ApControl(ComponentFactory aFactory) {
      factory = aFactory;
   }
   void doRendering ( ) {
      factory.r.renderOpA(); factory.r.renderOpB();
      factory.r.renderOpC();
   }
   void doShading ( ) {
      factory.s.shadeOpA(); factory.s.shadeOpB();
      factory.s.shadeOpC();
   }
}
```

Gerekçe: önceki halde `ApControl`, fabrikadan aldığı renderer ve shader'ı ayrı üye alanlarda tutar. Texture Mapper gibi yeni bir özellik gelince `ApControl`'e yeni bir üye alan (`private TextureMapper t` gibi) ve kurucuya onu ilklendiren yeni bir satır eklemek gerekir. Hoca bunu, kodu gereksiz yere uzatan bir kod kusuru olarak niteledi. Fabrikanın tamamını tutmak bunu önler: yeni özellik fabrikada olur, `ApControl` ona `factory.` üzerinden erişir. Bu biraz daha fazla tuşa basmak gerektirebilir, ama önemli olan kodun anlaşılabilirliğini ve esnekliğini en üst düzeyde tutmaktır. Hoca bu halin, kod kusurları konusu görüldükten sonra hepsinin arasında en doğru çözüm olacağını söyledi.

> **Ders sonrası not:** Slayttaki `factory.r.renderOpA()` biçimi, `ComponentFactory`'nin `r` ve `s` adlı erişilebilir alanlar tutmasını gerektirir. Bu, önceki slaytlardaki yalnız `cre-`

`ateRenderer()` ve `createShader()` metotları olan `ComponentFactory` yapısından farklıdır ve derste ayrıntılandırılmadı.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın kataloğundaki Preserve Whole Object, bir nesneden birkaç değeri ayrı ayrı alıp başka bir yere geçirmek yerine nesnenin tamamını geçirmeyi önerir. Böylece ileride o nesneden başka bir değer gerektiğinde çağrı imzası ya da alan listesi değişmez. Bunun bedeli, çağıranın o nesneye bağımlı hale gelmesidir.

11. Çıkarılan Dersler ve Genel Tasarım Kuralları

Hoca yaklaşık iki saatlik deneyimi toparladı: bir problem farklı yollarla çözüldü ve çözümlerin birbirine göre avantaj ve dezavantajları incelendi. Çıkarılan dersler (02:32:25):

1. Neyin değiştiğini bul ve onu sarmala. Sarmalamak burada değişeni bir kalıtım hiyerarşisine koymak, yani soyut bir üst sınıf ve onun somut varyasyonları biçiminde soyutlamaktır. Böylece değişen şey kara kutu yaklaşımıyla kullanılabilir. Örnekte `ApControl` (kalıptaki client), hangi oyun motoru, ekran kartı ya da sürücü kütüphanesinin çalıştığını bilmeden renderer ve shader özelliklerini kullanabildi. Ona DirectX gerçeklemesi verilirse DirectX ile, OpenGL verilirse OpenGL ile çalıştı. İç ayrıntıları bilmeyi gerektiren beyaz kutu yaklaşımı yerine "bana render et" / "buyur" biçiminde bir etkileşim oldu.

Bunun bakımda da avantajı vardır: DirectX renderer'ında bir hata bulunduğunda (kütüphaneden değil kendi programımızdan kaynaklanan), kodun birçok yerini aramak yerine nereye müdahale edileceği nokta atışıyla bulunur. Bu tür kolaylıklar çabadan, zamandan ve paradan büyük tasarruf sağlar.

> **Hocanın vurgusu:** Ulusal ve uluslararası birçok araştırmaya göre bir yazılım projesine harcanan çabanın, zamanın ya da maliyetin ortalama %50'si bakım aşamasında harcanır: yazılımı hazır hale getirmek 50 birimse, hatalarını düzeltmek ve değişen gereksinimleri karşılamak da 50 birimdir. Bakımı kolaylaştıran her çaba (okunabilirlik, anlaşılabilirlik, kolay değişiklik) sonradan avantaj olarak geri döner.

2. Kalıtım yerine parça-bütün ilişkisini tercih et. Hoca bunun deneyimli tasarımcıların önemli bir kısmının izlediği yol olduğunu söyledi ve Bölüm 9.3'teki örneği bu ilkeyle ilişkilendirdi: oyunun tamamını kütüphaneye göre türetmek, oyun mantığındaki bir değişikliğin ya da hatanın her alt sınıfta ayrı ayrı düzeltilmesini gerektirir. Kalıtım burada yanlış bir amaç için kullanılmış olur.

3. Gerçeklemelere göre değil, arayüzlere göre tasarla. Böylece kara kutu yaklaşımı kullanılabilir. `ApControl` fabrikadan yalnız "bana bir renderer ver, bana bir shader ver" der ve "işini yap" diye çağırır. OpenGL'in ya da DirectX'in shader'ının nasıl kullanılması gerektiğine dair hiçbir kodu yoktur.

> **Ek bilgi (derste anlatılmadı):** "Program to an interface, not an implementation" ve "Favor object composition over class inheritance" ilkeleri GoF kitabının giriş bölümünde büyük olasılıkla nesneye dayalı tasarımın iki temel ilkesi olarak verilir. "Değişeni bul ve sarmala" ilkesi de büyük olasılıkla *Head First Design Patterns* ve *Design Patterns Explained* kitaplarında ilk tasarım ilkesi olarak öne çıkarılır.

Örnek Sorular

Soru 1 (ekrandan, 00:40:00): Abstract Factory'nin GUI örneğinde (Motif ve Presentation Manager görünüşleri) kalıp yerine `WidgetFactory.setLoookAndFeel( val: EnumLookFeel )` metodu ve içinde `switch( val ) { … }` kullanılırsa bu çözümün zayıf yönü nedir? Başka hangi seçenek düşünülebilir?

> **Çözüm (derste verildi):** Yeni görünüş biçimleri (örneğin GNOME, KDE) için enum'a yeni değerler ve switch içine yeni case komutları eklemek gerekir. Her case'e öncekiler kadar uzun (20-30 satır) kod blokları girer; bakım zorlaşır, angarya iş artar ve "Long Method" kod kusuru oluşur. Switch-case kullanmak, kalıtım ve çok biçimliliğe ihtiyaç olduğunun kuvvetli bir işaretidir; başka seçenek olarak soyut fabrika ve soyut ürünlerden oluşan bir kalıtım hiyerarşisi (kalıbın kendisi) kurulur.

Soru 2 (derste sözlü, 01:18:05): Oyun motoru örneğinde switch-case ile yapılan Çözüm 1'e DirectX'in yeni bir sürümü (DirectX 12) eklenmek istenirse nerelerde değişiklik yapmak gerekir?

> **Çözüm (derste verildi):** `LibraryType` enum'una yeni bir değer eklenir; `DirectXRenderer12` ve `DirectXShader12` sınıfları oluşturulup kodlanır; `ApControl`'e bu türlerden yeni üye alanlar eklenir; `doRendering` ve `doShading` metotlarının her birine yeni bir case eklenir. Toplamda iki yeni sınıf oluşturulur ve iki sınıfın (enum ve `ApControl`) koduna müdahale edilir. Texture mapping gibi her yeni yetenek bu işi her kütüphane ve sürüm için tekrarlatır.

Soru 3 (derste sözlü, 01:29:26 ve 01:53:46): Switch-case'ten kurtulmak için kalıtım kullanılırken hiyerarşi yanlış kurulursa (örneğin kütüphane başına `OpenGL`, `DirectX` sınıfları kurulup her biri kendi renderer ve shader'ına sahip olursa) ne olur?

> **Çözüm (derste verildi):** Ortaya kalıtım değil sahiplik ilişkisi çıkar; kütüphane sınıfları her render ve shade işlemini ilgili nesneye ileten delegate metotlarla dolar (`public void shadeOpA() { s.shadeOpA(); }`). DirectX 12 eklemek için üç sınıf (`DirectX12`, `DirectX12Renderer`, `DirectX12Shader`) gerekir; doğru kalıtımla iki kardeş sınıf yeterlidir. Araya soyut bir `GameEngine` konsa da üç sınıf gerekir. Bu tasarım switch-case çözümüne göre avantaj getirmez ve tercih edilmez.

Soru 4 (ekrandan, 02:30:00): Tüm çözümlerde library nesnesi `ApControl` kurucusuna parametre olarak verilirse daha doğru bir iş yapılmış olur. Neden? Bize ne kazandırır?

> **Çözüm (derste verildi):** Hangi kütüphanenin kullanılacağının seçimi (switch-case) koddan çıkar; kurucuyu çağıran yere, bir yapılandırma programına ya da yapılandırma dosyasına ötelenir. `ApControl` hangi aileyle çalıştığını hiç bilmez. Switch-case yine de bir yerde kalır, ama yalnız ilklendirme noktasına geri çekilir. Aynı şey Çözüm 2'de renderer ve shader nesnelerini (ya da `LibraryType` değerini) parametre vererek yapılabilir; bu açıdan kalıbın ayrıca bir avantajı yoktur.

Soru 5 (ekrandan, 02:31:40): Refactoring konusunu gördükten sonra Preserve Whole Object eyleminin yürütülmüş hali (`ApControl`'ün yalnız `ComponentFactory factory` tutması ve `factory.r.renderOpA()` biçiminde çağırması) tercih edilir. Neden? Bize ne kazandırır?

> **Çözüm (derste verildi):** Renderer ve shader ayrı üye alanlarda tutulursa, Texture Mapper gibi yeni bir özellik geldiğinde `ApControl`'e yeni bir üye alan ve onu ilklendiren yeni bir satır eklemek gerekir; bu, kodu gereksiz yere uzatan bir kod kusurudur. Fabrikanın tamamı tutulduğunda yeni özellik fabrikadan `factory.` ile kullanılır, `ApControl`'ün alan listesi değişmez. Biraz daha fazla yazmak gerekebilir, ama anlaşılabilirlik ve esneklik daha önemlidir.

Soru 6 (derste sözlü, 02:17:35): DirectX 12 desteği eklemek, kalıtımı doğru kullanan Çözüm 2'de mi, Abstract Factory'li Çözüm 3'te mi daha az sınıf gerektirir? Buradan ne sonuç çıkar?

> **Çözüm (derste verildi):** Çözüm 2'de iki kardeş sınıf (`DirectX12Renderer`, `DirectX12Shader`) yeterlidir. Çözüm 3'te bunlara ek olarak bir `DirectX12Factory` sınıfı da zorunludur, yani bir sınıf fazladır. Bu örnekte kalıplı çözümün doğru kurulmuş kalıtıma göre görünür bir avantajı yoktur; kalıp kullanmak her zaman bir gereklilik değildir. Ancak kalıtım hiyerarşisini doğru kurmak deneyim ister; yanlış kurulursa kalıplı çözümden daha kötü bir tasarıma varılır.

Dersin Toparlaması

Bu derste GoF'un ilk yaratımsal kalıbı Abstract Factory işlendi. Kalıbın amacı ilişkili nesne ailelerini somut sınıflarını belirtmeden oluşturmaktır; client yalnız soyut fabrika ve soyut ürünlerle çalışır, hangi ailenin kullanıldığını bilmez. Kitaptaki pencere yöneticisi örneği üzerinden kalıbın yapısı ve bileşenleri (AbstractFactory, ConcreteFactoryY, AbstractProductX, ProductXY, Client) anlatıldı. Hoca GoF'un eski gösterimli şemasını güncel UML'e çevirerek «`abstract`» stereotipi, `{abstract}` metotlar, görünürlük işaretleri, dönüş türleri ve kullanma ilişkileri gibi ayrıntıları hatırlattı. Switch-case'in, kalıtım ve çok biçimlilik eksikliğinin kuvvetli bir işareti olduğu vurgulandı.

Dersin ikinci yarısında hocanın kendi örneği olan üç boyutlu oyun motoru üzerinden dört yaklaşım karşılaştırıldı: switch-case, doğru kurulmuş kalıtım, yanlış kurulmuş kalıtım/sahiplik ve Abstract Factory. Her birine DirectX 12 desteği eklemenin maliyeti incelendi. Sonuçta switch-case'in hiçbir çözümde tamamen yok olmadığı, yalnız ilklendirme noktasına çekildiği ve bu örnekte kalıbın doğru kalıtıma göre ek bir avantaj sağlamadığı görüldü. Ders üç genel kuralla kapandı: değişeni bul ve sarmala, kalıtım yerine parça-bütün ilişkisini tercih et, gerçeklemelere göre değil arayüzlere göre tasarla. Bir sonraki derste, adı benzese de farklı bir problemi çözen Factory Method kalıbı işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.